

Physical Agent 开发计划

当前开发重点

下一阶段重点不是直接堆叠更多外部视觉工具，而是先把评测协议、坐标映射和 QA 视图选择固定下来，再在同一套 TaskProfile 上开发 Planner、Router、Executor、Verifier 的路线化能力。当前优先级应按“评测可复现 -> 规划可结构化 -> 路由可解释 -> 执行可控 -> 验证可阻断 -> 失败可回流”推进。

参考调研笔记: [tool_router_physics_tool_research.md](#)

目标架构

User image + instruction

-> Planner: 生成 task profile 和初始 edit prompt

-> Router: 根据编辑动作、物理依赖、空间范围选择 route/toolchain

-> Tool Executor: direct edit / mask edit / detection / segmentation / inpainting / QA

-> Verifier: VLM 分项评分 + PICABench QA + deterministic guardrails

-> pass: accept

-> fail: 生成 repair instruction, 并带历史反馈重新 route/edit

流程图对应的当前代码实现

本节根据 [references/工作流.png](#) 的组件顺序，记录当前代码实际已经采用的实现方法。它描述的是当前 baseline，不等于后续计划方案；后续优化应以本节为代码事实基础，避免把未实现的 route/tool/mask 能力误写成当前能力。

workflow 组件	当前代码位置	当前实现方法	当前输入	当前输出/产物	主
User Image + Instruction	scripts/run_picabench_examples.py 、 scripts/run_picabench_resumable.py	从 PICABench manifest 读取 input_png 、 instruction 、 physics_category 、 physics_law 、 edit_operation 、 edit_area 、 annotated_qa_pairs ；运行前把原图标准化成 canonical input	原始 PNG、编辑指令、PICABench 标签和 QA 元数据	canonical_input.png 、 coordinate_transform.json 、传入 agent 的 eval_case metadata	目通认
图像标准化/坐标协议	src/physical_agent/pica_eval.py	使用 prepare_canonical_input() 把原图按 1024x1024 contain + padding 渲染到统一画布；用 contain_transform() 保存 scale/padding；评测时通过 source_box -> canonical_box -> output_box 映射 crop	原始图、目标 canonical size、PICABench edit/crop box	canonical 图、坐标变换 JSON、mapped crop、去 padding 的 final image	只动
MLLM Analyzer / Planning	src/physical_agent/planner.py	plan_edit() 调用兼容 OpenAI chat completions 的 VLM，要求返回严格 JSON；system prompt 指定字段 target 、 operation 、 preserve 、 physics_dependencies 、 route 、 edit_prompt 、 verifier_focus ；失败重试时把上一轮 verifier failure 拼入 prompt	当前图像、用户指令、上一轮失败说明	plan.json 中的自由结构 JSON 和 edit_prompt	还TaPl则
Tool Selection / Router	src/physical_agent/router.py	select_route(plan) 读取 plan 中的 route 字段，只接受 direct_edit 或 local_edit ；若请求 local_edit ，由于没有 mask/local CV 工具，直接回退到 direct_edit	Planner 输出的 plan	{"route": "direct_edit", "reason": ...}	尚caRe没
Tool Retrieval	当前没有独立模块	流程图中的工具检索节点尚未实现；当前系统没有工具库检索、工具注册表或动态工具选择，只在 Router 中硬编码回退原因	无	无	缺任ec工
Tool Executor / Generation/Edit	src/physical_agent/editor.py 、 src/physical_agent/openai_compat.py	execute_edit() 校验输入 PNG 后调用 OpenAICompatClient.edit_image() ；底层向 /images/edits 发 multipart 请求，默认 size="1024x1024"，取 data[0].b64_json 写为 candidate.png	canonical input、Planner 生成的 edit_prompt 、image edit model	每轮 step_xx/candidate.png	只mbc
Verification Agent	src/physical_agent/verifier.py	verify_edit() 同时看 original/candidate 两张图，结合 instruction 、 plan 、 route 、 metadata 生成 task profile 和 required checks；VLM	原图、候选图、指令、plan、route、metadata	verify.json 、 required_checks 、 blocking_failures 、	ch断avdi

工作流组件	当前代码位置	当前实现方法	当前输入	当前输出/产物	主
		输出 <code>pass</code> 、三项分数、 <code>check_results</code> 、 <code>blocking_failures</code> 、 <code>route_hints</code> ; <code>normalize_verification_result()</code> 将 <code>required/blocking failure</code> 作为硬门槛		<code>repair_instruction</code> 、 <code>route_hints</code>	
Pass / Fail Control	<code>src/physical_agent/orchestrator.py</code>	<code>run_agent()</code> 组织 Planner -> Router -> Editor -> Verifier 循环; 若 <code>verification["pass"]</code> 为真则接受并停止; 否则把 <code>repair_instruction</code> 或 <code>issues</code> 作为 <code>previous_failure</code> 进入下一轮; 最大轮数由 <code>Settings.max_retries</code> 控制	每轮 <code>verification</code> 结果、 <code>settings</code>	<code>run_state.json</code> 、 <code>iterations</code> 、 <code>accepted</code> 、 <code>final_image</code> 、调用次数统计	re 会 m
PICABench QA / Dataset Evaluation	<code>src/physical_agent/pica_eval.py</code> 、运行脚本	agent 完成后, <code>evaluate_picabench_case()</code> 对 <code>annotated_qa_pairs</code> 做 yes/no VLM 评测; <code>classify_question()</code> 用关键词把问题分成 <code>global_position</code> 、 <code>local_appearance</code> 、 <code>mixed</code> 、 <code>unknown</code> , 并选择 full image/crop/mixed 视图; 同时计算 non-edit PSNR	去 padding 的 final image、原图、QA pairs、 edit_area、 metadata	<code>pica_eval.json</code> 、逐题 QA 结果、 <code>accuracy</code> 、non-edit PSNR	Q PI 前
实验汇总与可视化	<code>scripts/run_picabench_examples.py</code> 、 <code>scripts/run_picabench_resumable.py</code> 、 <code>scripts/view_picabench_results.py</code>	批量脚本写 <code>summary.json</code> 和 <code>summary_metrics.json</code> ; resumable 脚本支持按 case 重试和 resume; Gradio viewer 支持按 physics law/category/operation 过滤并查看原图、结果图、QA 表和 crop gallery	run outputs、 manifest、 <code>pica_eval.json</code>	<code>summary</code> 、 <code>metrics</code> 、可视化界面	汇 ro bl
配置与 API 适配	<code>src/physical_agent/config.py</code> 、 <code>src/physical_agent/openai_compat.py</code> 、 <code>src/physical_agent/image_io.py</code>	从 <code>.env</code> 或环境变量读取 base URL、API key 和模型名; 用 <code>urllib</code> 实现 chat JSON、image edit multipart、重试; 图像 I/O 只接受有效 PNG, 并用 data URL 发送给 VLM	<code>.env</code> 、环境变量、PNG 文件	<code>Settings</code> 、API JSON、base64 图像文件	依 没 si

当前代码的整体执行链可以概括为：

```
PICABench case
-> canonicalize image and save coordinate transform
-> run_agent()
    -> plan_edit(): VLM JSON plan
    -> select_route(): direct_edit fallback
    -> execute_edit(): whole-image edit API
    -> verify_edit(): VLM route-aware checklist gate
    -> retry with previous_failure when verifier fails
-> write unpadded output
-> evaluate_picabench_case(): mapped QA crops/full-image QA + non-edit PSNR
-> summary.json / summary_metrics.json / Gradio viewer
```

因此，当前系统已经实现了工作流图中的主循环骨架、VLM planner、基础 router、图像编辑 executor、route-aware verifier、失败重试和 PICABench 离线评测；尚未实现的是独立工具检索、真实工具库、mask/local edit 工具链、基于三类物理标签的规则 planner/router、以及结构化 route hints 驱动的重试策略。

重新安排后的开发顺序

前面原有的五个阶段已经不再适合作为当前路线图：它们过早把 Router 和外部工具放在第一位，但从现有实验和尺寸问题暴露出的风险看，真正的优先级应先稳定评测协议和坐标协议，再逐步让 Planner、Router、Executor、Verifier 消费同一套结构化任务表示。新的开发顺序按“评测可复现 -> 决策可解释 -> 执行可控 -> 验证可阻断 -> 失败可回流”的闭环安排。

阶段 0：固定评测协议与坐标协议

目标：先保证后续优化的收益可以被稳定测到，避免 route 或工具改动被尺寸、crop、QA 视图选择等评测噪声掩盖。

当前应优先完成和保持的内容：

- 1. 统一生成 `canonical_input.png`，并记录 `coordinate_transform.json`。
- 2. 评测时使用 `source box -> canonical box -> output box` 的显式映射，而不是假设输出图与原图同尺寸。
- 3. 对 PICABench QA 按问题类型分流：全局位置关系使用完整图，局部外观/物理细节使用 mapped crop，混合问题同时提供完整图和 crop。
- 4. 在 `pica_eval.json` 中记录 `evaluation_view`、`source_box`、`mapped_box`、`output_size`、`context_risk`。
- 5. 将 non-edit PSNR、QA accuracy、accepted/pass 等指标都绑定到同一套坐标协议上。

完成标准：同一个 case 在不改 agent 行为时重复评测应得到稳定结果；低分 case 能区分是编辑失败、物理失败、尺寸映射失败，还是 QA 视图选择不当。

阶段 1: Planner 生成结构化 TaskProfile

目标: 让 Planner 不只是生成自然语言 `edit_prompt`, 而是生成 Router、Executor、Verifier 都能消费的结构化任务画像。

开发内容:

1. 实现 `LabelPolicyCompiler`: 把 PICABench 的 `physics_category`、`physics_law`、`edit_operation` 编译成规则骨架, 包括物理依赖、候选 route、must-pass checks、默认风险等级。
2. 实现 `VisionPlanner`: 让 LLM/VLM 读取图像和指令, 在规则骨架约束下补全 `target_objects`、`affected_objects`、`dependent_regions`、`reference_cues`、`edit_prompt`。
3. 实现 `PlanValidator`: 校验字段完整性, 禁止缺失关键物理依赖, 补全默认 `preserve scope` 和 `verifier focus`。
4. 固定 TaskProfile schema, 使后续组件不再解析自由文本。

选择原因: PICABench 标签是确定输入, 不应交给 LLM 重新猜测; 但图像中的对象、接触点、反射面、阴影承接面等细节必须由视觉模型补全。因此 Planner 应采用“规则骨架 + LLM/VLM 视觉填充 + 规则校验”的混合方案。

阶段 2: Router 与 ToolSpec/RouteSpec

目标: 把 TaskProfile 转换成明确的执行路线, 而不是让所有任务都退化成 whole-image direct edit。

开发内容:

1. 建立 `RoutePolicy`: 根据 `physics_category` 做一级分流, 根据 `physics_law` 选择机制路线, 根据 `edit_operation` 决定路线内部顺序。
2. 建立 `RouteDecision` 输出: 包含 `route`、`confidence`、`required_tools`、`optional_tools`、`requires_mask`、`mask_type`、`fallback_route`、`verification_focus`。
3. 定义 `ToolSpec`: 记录工具输入、输出、适用 law/operation、成本、风险和失败模式。
4. 定义 `RouteSpec`: 记录每条 route 的适用任务、工具链、必检项、fallback 和 retry hints。
5. 优先实现规则路由: `reflection_route`、`shadow_projection_route`、`causal_settle_route`、`direct_global_edit`、`localized_inpaint`、`qa_eval_only`。

选择原因: Router 的作用不是“聪明地写 prompt”, 而是把物理标签和 TaskProfile 转成可审计的工具链选择。尤其是 Reflection、Light Propagation、Causality 这类低分类型, 需要通过 route 明确要求反射、阴影、支撑关系等依赖区域同步更新。

阶段 3: Executor 路线化执行

目标: 让执行层真正体现 route 差异, 逐步从单一 API direct edit 过渡到局部、两阶段和候选生成策略。

开发内容:

1. 为每条 route 准备 route-specific prompt template, 而不是复用同一个通用编辑 prompt。
2. 对低风险任务保留 `one_pass_direct_edit`。
3. 对 Reflection、Light Propagation 等依赖效果任务支持 `two_stage_edit`: 先完成主对象变化, 再补齐反射/阴影/光照效果。
4. 对 Causality 支持 `causal_settle_route`: 先识别被移除/移动对象是否是支撑或约束, 再要求受影响对象进入稳定姿态。
5. 对高风险任务预留 `best_of_n`, 但应在 verifier 稳定后再扩大候选数。
6. 记录 `execution_trace`: 实际 route、工具调用、输入输出路径、fallback、候选数量和失败原因。

选择原因: 当前失败样本中, “摩托车放倒”“卡牌取走”这类任务不是单纯局部纹理编辑失败, 而是执行层没有显式模拟干预后的物理后果。Executor 必须把 route 中的物理依赖变成执行约束。

阶段 4: Route-aware Verifier 与 PICA Eval Gate

目标: 让 Verifier 不再只给通用分数, 而是按 route 检查必须满足的物理条件, 并能阻断明显物理错误的候选。

开发内容:

1. Verifier 输入 `route`、TaskProfile、metadata 和 PICABench QA 信息。
2. 根据 `physics_law` + `edit_operation` + `route` 生成 law-specific checklist。
3. 把检查项分成 `must_pass` 与 `soft_score`, `must-pass` 失败时即使总分较高也不能 accept。
4. 对涉及全局位置关系的问题使用完整图, 对局部物理细节使用 mapped crop, 对混合问题同时使用两种视图。
5. 输出 `check_results`、`blocking_failures`、`route_hints`、`required_checks`, 供 retry loop 消费。

选择原因: 低分样本往往不是“整体看起来不好”, 而是违反了某条关键物理关系, 例如反射缺失、阴影方向不一致、支撑物移除后主体仍悬空。Verifier 必须能把这些失败显式化。

阶段 5: Retry Loop 与实验闭环

目标: 让失败信息不只是自然语言 `repair prompt`, 而是能回落到 Planner/Router/Executor 的结构化修复信号。

开发内容:

1. 将 `blocking_failures` 转换成 `route_hints`, 例如 `missing_reflection`、`shadow_not_reprojected`、`support_removed_without_settle`。
2. Retry 时允许 Router 根据失败类型切换 route 或升级工具链。
3. 对高风险任务在 verifier 稳定后启用 `best_of_n`。
4. 实验汇总按 `physics_category`、`physics_law`、`edit_operation`、`route`、`question_type`、`context_risk` 分组。
5. 固定 regression set, 优先追踪 Reflection、Light Propagation、Causality 中表现最差的组合。

选择原因：如果 retry 只是把错误描述拼回 prompt，agent 很难稳定改正路线级错误。失败必须结构化，才能判断下一轮是补 mask、换 route、扩大 crop、增加完整图验证，还是直接拒绝当前候选。

阶段 6：真实视觉工具接入与路线扩展

目标：在评测协议、Planner/Router/Verifier 都稳定后，再接入 detector、segmenter、mask editor、inpainting 等更重的工具。

开发内容：

- 1. 先接轻量工具：coordinate mapper、law-specific verifier、mask generator stub、dependency region expander。
- 2. 再接真实视觉工具：object detector、segmenter、local mask editor、inpainting。
- 3. 对每个新工具补齐 ToolSpec、输入输出文件约定、失败模式和 fallback。
- 4. 每接入一个工具，都必须用固定 regression cases 验证它是否提升对应 route，而不是只看单例视觉效果。

选择原因：重工具会引入新的误检、mask 漂移、成本和运行不稳定问题。只有在前面的决策、执行和验证协议固定后，工具收益才能被准确归因。

开发原则

- 每个新工具必须有 ToolSpec，不能只在代码里硬编码调用。
- 每个 route 必须说明适用物理类型、输入要求、失败模式和 fallback。
- VLM 判断必须尽量拆成可解释分项；重要失败不能只藏在 rationale。
- PICABench 样例优先作为 regression cases。
- API key 仍只从本地 .env 或环境变量读取，不写入源码、文档或日志。

待确认

- 是否允许安装或运行 GroundingDINO、SAM/SAM2、LaMa 等本地模型。
- 是否优先使用闭源 API 的 mask edit 能力，还是先实现本地 mask/inpaint 接口。
- PICABench 评测是否只做定性展示，还是需要批量表格和可量化分数。
- 当前 gpt-image-2 输出尺寸默认 1024x1024，需要决定是请求原图比例、后处理回原尺寸，还是在 verifier 中判失败。

模块实际方案与计划方案对照

本节用于把当前已经落地的实现和下一步计划开发的方案放在同一张表里，便于汇报时区分“已经完成的 MVP 能力”和“后续需要研发的 agent 能力”。该内容适合放在开发计划文档中，而不是 full run results 或 progress report 中：full run results 主要记录实验数据，progress report 主要记录阶段性结论，development plan 则负责维护模块路线图。

模块	当前实际方案	当前局限	计划开发方案
数据集输入	使用 physical_image_editing_agent/data/picabench_examples/manifest.json 中的 50 条 PICABench 子集，运行时读取 explicit_prompt、input_png、edit_area 和 annotated_qa_pairs	样本规模较小；主要用于 MVP 验证；输入原图尺寸不统一	保持 50-case 子集作为 regression set，同时支持扩展到更多 stratified samples；记录每次实验的 manifest、prompt level 和 coordinate transform
图像标准化	新增 canonical_input.png 预处理：将原图按 1024x1024 contain + padding 标	padding 可能改变视觉边界；旧实验结果没有 transform 元数据	将 canonical transform 作为所有评测和后续工具路由的统一坐标协议；后续可支持不同 canonical size，但必须显式记录
Planner	调用 VLM 根据图像和 instruction 生成 JSON，包括 target、operation、preserve、physics_dependencies、route、edit_prompt、verifier_focus	physics_dependencies 仍是自由文本；route 基本不被真正执行；缺少可被工具消费的目标/依赖区域结构	升级为“规则骨架 + LLM 视觉填充 + 规则校验”的 TaskProfile：规则注入 physics_law、physical_operation、must_pass_checks，LLM 填充 target_objects、dependent_regions、reference_cues、edit_prompt
Router	当前 select_route(plan) 只接受 direct_edit/local_edit，但 local_edit 会回退到 direct_edit	所有任务实际走同一条 whole-image edit 路线，无法体现不同物理 law 的差异	实现 rule-based routing：reflection_route、shadow_projection_route、causal_settle_route、direct_global_edit、localized_inpaint、qa_eval_only；每个 route 定义输入、工具链、失败模式和 fallback
Executor	使用闭源 image edit API 对整图进行编辑，输出 candidate.png	缺少 mask、检测、分割、局部回贴；小目标和几何精确位置不稳定	接入局部工具链：detection/segmentation/mask edit/inpainting；对高风险任务支持两阶段编辑和 best-of-N candidate selection

模块	当前实际方案	当前局限	计划开发方案
Verifier	使用 VLM 检查 instruction、preservation、physics，并输出 pass、分数、issues 和 repair instruction	通用 verifier 容易漏检 law-specific 失败；已有 case 出现 agent accepted 但 PICAEval 低分	建立 law-specific checklist；将 PICABench QA、尺寸/格式 guardrail、非编辑区一致性和 VLM 分项评分合并为 accept gate
PICAEval QA	已从单一 crop 评测升级为按 question_type 分流：全局位置用完整图，局部物理用 mapped crop，混合问题用完整图 + crop	QA 类型目前由启发式关键词分类，可能误分；mixed 问题成本更高	为 QA 增加稳定分类字段；统计 context_risk；后续可用少量人工标注或 LLM classifier 校正问题类型
坐标映射	新增 source_box -> canonical_box -> output_box 显式映射；pica_eval.json 记录 source_box、mapped_box、output_size、evaluation_view	只能处理 contain/identity 协议；无法自动纠正生成模型内部构图漂移	将 transform 写入 run state；若后续 API 或工具产生 crop/resize/paste，需要把每一步 transform 组合成可追踪链路
Non-edit PSNR	使用同一套坐标协议把 edit_area 映射到输出图，基于 canonical/source canvas 计算非编辑区 PSNR	对全局状态转移任务，低 PSNR 不一定代表失败；padding 区域可能影响解释	按任务类型解释 PSNR：global state 单独统计；local/object edit 作为保真约束；后续引入 SSIM/LPIPS 或 mask-aware preservation
Retry Loop	当前失败后把 verifier 的 repair_instruction 传回 planner，再重新编辑	重试只是自然语言修复，route 不会根据失败类型调整；高风险任务没有更高候选预算	将失败项转为 route hints；对 Reflection add/move、Light_Propagation move、Causality support-removal 设置更高重试或 best-of-N
实验汇总	summary.json 和 summary_metrics.json 记录每个 case 的 accuracy、PSNR、accepted、attempts	旧汇总未记录 QA 视图、坐标映射、context risk；难以区分真实失败和评测噪声	新实验汇总加入 transform 路径、canonical input、question type 分布、context-risk accuracy，并按 law/route/operation 联合分析

短期执行顺序：

- 1. 先固定评测协议：canonical input、坐标映射、QA 视图分流、PSNR 坐标统一。
- 2. 再重跑重点 regression cases，尤其是 picabench_0816_light_propagation、picabench_0387_reflection、picabench_0294_causality。
- 3. 如果低分仍集中在 Reflection 和 Light_Propagation，再开发 route-specific planner/verifier。
- 4. 最后接入 mask/detection/segmentation 等工具，避免在评测协议未稳定前优化错误目标。

PICABench 三类标签在模块中的作用

PICABench 每个样本提供 physics_category、physics_law、edit_operation 三类标签。它们不应只作为统计字段，而应分别承担不同粒度的 agent 决策作用。

标签	粒度	主要作用	适合驱动模块
physics_category	粗粒度物理域	判断任务属于 Optics、Mechanics 还是 State，决定整体问题范式	Planner 总体分析、Router 一级分流、报告统计
physics_law	中粒度物理机制	判断具体依赖机制，例如 Reflection、Light_Propagation、Causality、Global、Local	route 选择、law-specific checklist、dependent region 规划
edit_operation	动作类型	判断用户要求是 add/remove/move/replace/weather/time/wet 等哪种编辑动作	prompt 改写、工具链选择、失败模式预测、重试策略

三者组合后才能形成真正可执行的 agent policy。例如：

```
physics_category = Optics
physics_law = Reflection
edit_operation = add

=> 不是普通 add_object，而是 reflection_add_route:
    add target object + add reflection + align waterline/mirror plane + verify scale and contact.
```

再例如：

```
physics_category = Mechanics
physics_law = Causality
edit_operation = remove

=> 不是普通 remove_object，而是 causal_settle_route:
    remove support + infer affected object final pose + update contact/shadow/occlusion.
```


模块使用方式如下。

模块	使用 physics_category	使用 physics_law	使用 edit_operation
Planner	决定解释框架：光学依赖、力学因果或状态转移	展开具体依赖项，如 shadow/reflection/refraction/contact/material state	把普通动作改写成物理动作，如 remove_support_and_settle、move_and_reproject_shadow
Router	一级分流到 Optics/Mechanics/State 路线族	选择具体 route，如 reflection_route、shadow_projection_route、causal_settle_route	决定 route 内部工具顺序，如 add 先插入主体再补效应，remove 先删主体再修依赖
Executor	控制编辑范围：全局状态任务可整图编辑，局部/对象任务优先局部编辑	决定是否需要依赖区域，如反射面、阴影承接面、支撑点	决定调用 direct edit、mask edit、inpaint、move/deform 或二阶段编辑
Verifier	选择评分侧重点：光学、力学、状态一致性	使用 law-specific checklist；不同 law 的 pass 条件不同	检查动作是否完成，以及动作后果是否同步出现
PICAEval	按 category/law 汇总指标	按 law 定位薄弱类型，如 Reflection 和 Light_Propagation	按 operation 找失败组合，如 Reflection+add、Light_Propagation+move
Retry	粗粒度决定是否允许大范围重写	根据 law 生成 repair focus	根据 operation 调整下一轮策略，例如 add 失败先保证目标可见，move 失败先保证位置正确

这套标签设计与物理推理研究中的常见分解是一致的：物理任务通常不能只按视觉语义分类，而要同时表示物理机制、动作干预和可观测后果。对于本项目，physics_law 负责机制，edit_operation 负责干预，physics_category 负责高层问题域；三者合起来才能定义 route、plan 和 verifier。

组件级使用细则

下面进一步把三类标签落实到每个 agent 组件的输入、决策和输出中。

Planner

Planner 的目标输出通过混合方案得到：规则先给出不可漏的物理骨架，LLM/VLM 再根据图像填充具体对象和空间细节，最后由规则校验器补全和规范化。也就是说，目标输出主要由 LLM 生成具体内容，但必须由规则模板驱动和校验；PICABench 的三类标签是规则层的确定输入。

```
LabelPolicyCompiler(category, law, operation)
-> policy scaffold
VisionPlanner(image, instruction, policy scaffold)
-> draft task profile
PlanValidator(draft, policy scaffold)
-> final task profile
```

标签	Planner 中的具体用法	输出字段
physics_category	规则选择推理模板：Optics 关注光路和视觉依赖，Mechanics 关注支撑/接触/形变，State 关注状态范围和材料变化	reasoning_template、edit_scope
physics_law	规则展开必须同步处理的物理依赖。例如 Reflection 展开 reflection surface / reflected object / waterline；Causality 展开 support / affected object / final stable pose	dependent_effects、dependent_regions、must_pass_checks
edit_operation	规则把普通编辑动作改写成带物理后果的动作。例如 remove + Causality 改写成 remove_support_and_settle	physical_operation、expected_stable_outcome

LLM/VLM 负责填充：

字段	LLM/VLM 任务
target_objects	从图像和指令识别具体目标
affected_objects	找出受物理后果影响的对象
dependent_regions	描述阴影、反射、支撑点、接触面、状态变化边界等区域
reference_cues	找已有参考线索，如参考阴影、光源方向、地面平面、水线、材质边界
edit_prompt	把规则骨架和图像细节合成可执行编辑指令

规则 validator 负责检查：

检查	例子
必备字段不为空	Reflection 必须有 reflected region 或 reflective surface
物理动作一致	Causality + remove 必须生成 remove_support_and_settle
checklist 完整	Light_Propagation 必须检查 shadow direction、visibility、softness
route 合法	禁止 LLM 随意输出未注册 route
prompt 包含关键后果	支撑移除不能只写 remove target，必须写 final stable outcome

Planner 目标输出示例:

```
{
  "operation": "remove",
  "physical_operation": "remove_support_and_settle",
  "physics_category": "Mechanics",
  "physics_law": "Causality",
  "edit_scope": "object_plus_dependent_regions",
  "target_objects": ["motorcycle kickstand"],
  "dependent_effects": ["motorcycle final pose", "left-side contact shadow", "road contact area"],
  "reference_cues": ["gravity direction", "road plane", "existing low backlight"],
  "expected_stable_outcome": "motorcycle rests on its left side after kickstand removal",
  "must_pass_checks": [
    "kickstand absent",
    "motorcycle not upright",
    "new contact area visible",
    "continuous contact shadow present"
  ]
}
```

Router

Router的下一步优化目标是: 从当前的 `direct_edit` fallback 变成“标签组合 + TaskProfile + 工具能力表”的确定性路由器。Router 可以参考 LLM 生成的 plan, 但不能盲信 Planner 自报的 `route` 字段; 最终 `route` 应由规则和工具可用性共同决定。

推荐流程:

```
TaskProfile
-> RoutePolicy: 根据 category/law/operation 生成候选 routes
-> CapabilityMatcher: 检查当前工具是否满足 route 输入要求
-> RiskScorer: 根据任务风险、历史失败和操作类型设置 retry/candidate budget
-> RouteValidator: 确认 fallback、verification_focus、required_inputs 完整
-> RouteDecision
```

Router 输出不应只是 `route` 名字, 而应是完整决策对象:

```
{
  "route": "shadow_projection_route",
  "route_family": "Optics",
  "reason": "Light Propagation tasks require target movement plus shadow reprojection.",
  "required_inputs": ["target_object", "reference_shadow", "receiving_surface"],
  "available_tools": ["direct_edit", "pica_eval", "vlm_verifier"],
  "missing_tools": ["shadow_detector", "mask_editor"],
  "toolchain": ["direct_edit", "shadow_specific_verifier"],
  "fallback_route": "direct_edit_with_shadow_constraints",
  "retry_budget": 2,
  "verification_focus": ["target position", "old shadow removal", "new shadow direction", "shadow softness"]
}
```

标签组合到 `route` 的初始规则:

条件组合	推荐 route	初始工具链	缺少工具时的 fallback
Optics + Reflection + add/move	reflection_add_or_move_route	two-pass edit: add/move主体 -> 补反射/水面接触	direct_edit_with_reflection_constraints
Optics + Reflection + remove	reflection_remove_route	删除主体 + 删除镜像/高光/扰动	direct_edit_with_reflection_cleanup
Optics + Light_Propagation + move/add/remove	shadow_projection_route	编辑主体 -> 重投影 cast/contact shadow -> shadow verifier	direct_edit_with_shadow_constraints
Optics + Refraction + remove/replace	refraction_reconstruction_route	移除/改变介质 -> 重建未折射背景	direct_edit_with_refraction_reconstruction
Optics + Light_Source_Effects + add/replace	light_source_route	添加/替换光源 -> 局部亮度、色温、阴影、反射更新	direct_edit_with_light_falloff_constraints
Mechanics + Causality + remove	causal_settle_route	删除支撑 -> 生成最终稳定态 -> 接触/阴影更新	direct_edit_with_stable_outcome_constraints

条件组合	推荐 route	初始工具链	缺少工具时的 fallback
Mechanics + Deformation + move/others	deformation_route	形变/姿态编辑 -> 材料和纹理连续性检查	direct_edit_with_material_constraints
State + Global + weather/time/season	direct_global_edit	整图编辑 -> 全局状态 verifier	无需局部 fallback
State + Local + wet/frozen/burn/melt	localized_state_route	局部状态编辑 -> 边界/材料 verifier	direct_edit_with_locality_constraints

Router 的决策顺序：

1. 读取 `physics_category`，选择 route family：Optics、Mechanics、State。
2. 读取 `physics_law`，选择具体物理机制 route。
3. 读取 `edit_operation`，决定 route 内工具顺序。例如 `add` 先保证目标出现，`remove` 先清除目标及依赖效果，`move` 必须清理旧位置并生成新位置依赖。
4. 检查 TaskProfile 的 `dependent_effects`、`dependent_regions`、`reference_cues` 是否满足 route 的 `required_inputs`。
5. 检查工具能力表。如果缺少 mask/detector/segmenter，则选择明确的 direct-edit fallback，而不是假装走局部 route。
6. 设置风险等级和预算。Reflection add/move、Light_Propagation move、Causality support-removal 属于高风险，应提高 retry 或 best-of-N。
7. 输出 `RouteDecision`，供 Executor 和 Verifier 使用。

工具能力表初始结构：

```
{
  "direct_edit": {
    "available": true,
    "provides": ["whole_image_edit"],
    "risks": ["layout drift", "non_edit_region_change"]
  },
  "mask_editor": {
    "available": false,
    "provides": ["localized_edit"],
    "required_inputs": ["mask"]
  },
  "detector": {
    "available": false,
    "provides": ["object_box"],
    "required_inputs": ["text_query"]
  },
  "pica_eval": {
    "available": true,
    "provides": ["region_grounded_qa"]
  }
}
```

Router 初期实现应是规则优先，不急于使用 LLM 选择工具。LLM 可以给 route hints，但 route 的最终选择必须可复现、可解释，并记录触发规则。这样做的原因是：当前 full run 的主要失败集中在 Reflection 和 Light_Propagation，问题不是模型完全不知道物理规则，而是通用 `direct_edit` 没有把物理依赖转化为强执行约束。Router 的价值就是把标签和 TaskProfile 变成不同执行路线、不同 verifier focus 和不同重试预算。

Router 的核心作用可以概括为三点：

1. 把“任务是什么”转成“应该怎么执行”。Planner 负责理解图像和生成 task profile，Router 负责决定执行路线。
2. 把 `physics_category`、`physics_law`、`edit_operation` 三类标签组合成可执行策略，而不是把所有任务都交给 `direct_edit`。
3. 把后续 Verifier 的关注点一起确定下来。不同 route 失败模式不同，因此验收 checklist 也必须不同。

三类标签在 Router 中的决策过程：

```
physics_category: 先判断大方向
Optics    -> 重点处理光、影、反射、折射、光源
Mechanics -> 重点处理支撑、接触、重力、形变、稳定态
State     -> 重点处理全局/局部状态范围和材料变化

physics_law: 再判断具体机制
Reflection -> 需要主体和镜像/水面反射联动
Light_Propagation -> 需要阴影投射和遮挡关系重算
Causality  -> 需要干预后的物理后果
Global     -> 需要全图一致状态变化
Local      -> 需要局部状态改变且边界受控

edit_operation: 最后判断干预动作
add         -> 先保证新对象出现，再补物理效应
remove      -> 删除目标，同时删除或更新依赖效应
move        -> 清理旧位置，在新位置重建依赖效应
replace     -> 保留结构位置，但重算材质、光照或反射
```


具体例子：

Optics + Reflection + add

这不是普通“添加一艘船”。Router 应选择 `reflection_add_or_move_route`，原因是水面/镜面任务的成功条件包括：船本体可见、倒影在正确位置、倒影尺度与方向合理、水线接触成立、微波纹或接触暗带存在。若只走 `direct_edit`，模型可能加了船但没有反射，或反射与水线脱节。

Optics + Light_Propagation + move

这不是普通“移动花盆”。Router 应选择 `shadow_projection_route`，原因是移动物体后旧阴影应消失，新阴影应根据主光源、承接面和参考阴影重投影。若只走 `direct_edit`，常见失败是目标移动了，但阴影仍在旧位置，或新阴影方向/软硬不符合场景。

Mechanics + Causality + remove

这不是普通“删除支架”。Router 应选择 `causal_settle_route`，原因是删除支撑物会改变受力结构。成功结果不是支撑物消失，而是受影响物体进入最终稳定态，例如倾倒、下落、旋转或形成新的接触面。

State + Global + weather

这类任务适合 `direct_global_edit`，因为天气/季节/昼夜变化天然影响整张图：天空、地面、植被、光照、阴影、空气能见度都应一致改变。强行 `mask` 反而可能导致局部真实、全局不一致。

Route 对象特点与是否需要 mask

不同 route 的根本差异不是名字，而是它面对的对象类型、依赖区域和空间控制要求不同。

Route	适用对象特点	典型例子	是否需要 mask	原因
<code>direct_global_edit</code>	整个场景状态都应变化；没有单一小目标	白天变夜晚、晴天变雨天、夏季变冬季	通常不需要	全局状态需要整图一致变化，mask 会限制必要传播
<code>localized_inpaint</code>	小目标、局部删除/替换，背景需要强保持	删除桌上的杯子、去掉墙上一盏小灯、移除地面小物体	需要	目标小且背景应连续，mask 能减少非编辑区漂移
<code>reflection_add_or_move_route</code>	主体与反射面强绑定；常在水面、镜面、金属面上	湖面加船、移动水边小船、镜中新增物体	最好需要，至少需要区域约束	需要同时控制主体区域和反射区域；没有 mask 时主体可能不在标注区或倒影错位
<code>reflection_remove_route</code>	被删对象在反射面中也有对应痕迹	删除镜前的人、移除桌上反射在玻璃面上的杯子	需要或强烈建议	只 mask 主体不够，还要扩展到反射、高光和接触扰动
<code>shadow_projection_route</code>	目标和阴影承接面强绑定	移动花盆后重投影墙上阴影、删除物体及其地面影子	需要依赖区域 mask 或 expanded mask	阴影常不在主体 mask 内，必须覆盖 cast shadow/contact shadow
<code>light_source_route</code>	新光源影响周围表面、阴影和色温	加台灯、点燃蜡烛、替换发光屏幕	局部到半全局，视任务而定	光源影响范围超过目标本体；mask 过小会漏掉光照 falloff
<code>refraction_reconstruction_route</code>	透明介质遮挡并扭曲背景	移除玻璃杯、水位变化、去掉透明瓶	需要	需要重建被折射区域和介质边界，普通目标 mask 往往不覆盖全部扭曲背景
<code>causal_settle_route</code>	一个动作会改变支撑/接触/稳定状态	删除摩托车脚撑、移除桌脚、拿掉承重卡片	需要对象和受影响区域，通常不止一个 mask	既要删支撑物，又要编辑受影响主体姿态和新接触阴影
<code>deformation_route</code>	形状变化受材料属性约束	弯曲软管、压扁垫子、拉伸布料、改变椅子高度	通常需要	需要控制形变边界，避免刚体错误弯曲或纹理撕裂
<code>localized_state_route</code>	局部材料状态变化，边界需要受控	局部打湿、结冰、融化、烧焦、破裂	需要	状态变化应限制在局部区域，同时保留周边材质和构图

判断是否需要 mask 的实用规则：

如果编辑目标小、背景需要保持、或物理依赖区域不应扩散 -> 需要 mask。
如果任务影响整图语义和环境状态 -> 通常不需要 mask。
如果物理依赖区域不等于目标本体，例如阴影、反射、折射、接触痕迹 -> 需要 expanded mask 或多 mask。

三种常见 mask：

Mask 类型	覆盖对象	用途
target_mask	被添加/删除/移动的主体	控制主体编辑范围
dependency_mask	阴影、反射、折射区域、接触痕迹、受影响对象	让物理后果同步更新
preserve_mask	不应被改动的背景、人物、关键结构	保护非编辑区域

因此，Router 不只是决定“用不用 mask”，而是决定“需要哪几种 mask”。例如删除玻璃杯时，target_mask 覆盖玻璃杯本体，dependency_mask 覆盖桌面阴影、caustic、反射和杯后被折射的人脸/背景，preserve_mask 保护人物其他区域和桌面边缘。这个区分是物理一致性编辑区别于普通局部修图的关键。

Router 相关工具优化

Router 的优化不能只停留在“多几个 route 名字”，还必须同步优化工具抽象。否则 route 即使选对，也无法稳定执行。下一步工具优化的目标是建立一个轻量但可扩展的 tool inventory，让 Router 能根据任务标签和 TaskProfile 判断哪些工具可用、缺哪些输入、需要 fallback 到什么路线。

工具优化分为三层：

ToolSpec: 描述每个工具能做什么、需要什么输入、有什么风险
CapabilityMatcher: 判断某个 route 的 required_inputs 当前是否满足
RouteToolchain: 为每个 route 组合工具顺序和 fallback

ToolSpec 设计

每个工具都应有结构化描述，避免在代码里只用字符串硬编码。

```
{
  "name": "mask_editor",
  "capabilities": ["localized_edit", "inpaint"],
  "required_inputs": ["image", "mask", "edit_prompt"],
  "provided_outputs": ["candidate_image"],
  "physics_tags": ["Local", "Reflection", "Refraction", "Light_Propagation"],
  "cost": "medium",
  "risk": ["mask_boundary_artifact", "missed_dependency_region"],
  "fallback": "direct_edit"
}
```

Router 用这些字段判断：该 route 想用的工具是否可用，Planner 是否提供了足够输入，缺少输入时是否需要先调用 detector/segmenter，或者直接回退到 prompt-only direct edit。

优先建设的工具

优先级	工具	当前可实现方式	服务的 route	为什么优先
1	coordinate_mapper	已实现 source/canonical/output box 映射	qa_eval_only、所有评测 route	先保证评测和 crop 不错位，否则后续优化目标不可靠
2	law_specific_verifier	VLM + checklist + PICABench QA	Reflection、Light_Propagation、Causality	当前主要问题是 accepted 但 PICAEval 低分，需要先提高验收门槛
3	mask_generator_stub	先用 PICABench edit_area 和 QA box 生成候选 mask，不依赖外部模型	localized/reflection/shadow/refraction/causal routes	能先验证多 mask 协议，再决定是否接 SAM/GroundingDINO
4	dependency_region_expander	规则扩展 box：目标周围、下方阴影区、镜像区、水线区	shadow、reflection、refraction、causality	物理依赖区域常不等于目标本体，必须有 expanded mask
5	object_detector	后续接 GroundingDINO 或闭源 VLM box 输出	需要 target localization 的 routes	当前先不依赖外部模型，避免工具安装成本阻塞路线设计
6	segmenter	后续接 SAM/SAM2 或 API mask	local edit、remove、reflection cleanup	精细 mask 能减少非编辑区漂移，但需要 detector/point/box 输入
7	local_inpainter_or_mask_edit	API mask edit 或本地 inpainting	localized_inpaint、refraction reconstruction	真正执行局部编辑的核心工具，依赖前面 mask 质量
8	candidate_ranker	best-of-N + verifier score	高风险 route	解决生成随机性，尤其是 Reflection add/move 和 Causality settle

各 route 的工具需求

Route	必需工具	可选增强工具	暂时 fallback
direct_global_edit	direct_edit、 global_state_verifier	candidate_ranker	无，直接整图编辑
localized_inpaint	mask_generator、mask_editor、 preservation_checker	segmenter、object_detector	direct_edit_with_locality_constraints
reflection_add_or_move_route	direct_edit、 reflection_verifier、 coordinate_mapper	reflection_region_estimator、 mask_editor、candidate_ranker	direct_edit_with_reflection_constraints
reflection_remove_route	target_mask、dependency_mask、 mask_editor、 reflection_verifier	detector、segmenter	direct_edit_with_reflection_cleanup
shadow_projection_route	shadow_verifier、 dependency_region_expander	shadow_detector、 mask_editor、 reference_shadow_estimator	direct_edit_with_shadow_constraints
refraction_reconstruction_route	dependency_mask、 refraction_verifier	transparent_object_detector、 background_reconstruction	direct_edit_with_refraction_reconstruct
causal_settle_route	causal_decomposition_planner、 causal_verifier	support_detector、 contact_region_mask、 candidate_ranker	direct_edit_with_stable_outcome_constra
localized_state_route	target_mask、 local_state_verifier	segmenter、 material_boundary_detector	direct_edit_with_locality_constraints

先不用重工具的原因

短期不建议一开始就接 GroundingDINO、SAM2、LaMa 等完整本地工具链。原因是当前最紧迫的问题是 route 语义和评测协议：如果 verifier 仍然会接受明显失败样本，或者 crop 坐标仍然不可靠，加入更重的视觉工具也会优化错目标。

更稳妥的顺序是：

1. 先把 ToolSpec 和 route-toolchain 协议写清楚。
 2. 用 PICABench 标注框、edit_area 和规则扩展模拟 mask/dependency region。
 3. 用 law-specific verifier 判断 route 是否真的解决低分类类型。
 4. 确认 route 有收益后，再替换成真实 detector/segmenter/inpainter。

mask 工具的优化重点

对物理一致性编辑来说，mask 不是单一的“目标区域”。Router 应要求 mask 工具输出多种区域：

```
{
  "target_mask": "object to add/remove/move/change",
  "dependency_mask": "shadow/reflection/refraction/contact/affected object region",
  "preserve_mask": "regions that should remain unchanged"
}
```

不同物理 law 的 dependency mask 规则：

Law	dependency mask 应覆盖
Light_Propagation	cast shadow、contact shadow、受光/遮挡承接面
Reflection	倒影、高光、水线/镜面接触带、被主体遮挡的背景反射
Refraction	透明介质内部、被扭曲背景、caustic、边界高光
Causality	支撑点、受影响主体、新接触区域、可能倒落区域
Deformation	形变边界、纹理连续区域、相邻刚性结构
Local State	状态变化区域、材料边界、邻近湿痕/烟痕/结冰边缘

这也是 Router 和 Tool 的接口重点：Router 不应只问“有没有 mask”，而应要求工具返回与 physics law 对齐的 mask 类型，并把这些 mask 传给 Executor 和 Verifier。

Causality 专项：CLEVRER-style 因果分解

从 full run 结果看，Causality + remove 中的支撑移除任务表现不稳定。例如 picabench_0294_causality 中，系统删除了摩托车 kickstand，但 PICAEval 判断摩托车没有真正倒伏，也没有形成左侧连续接触阴影；移除承重卡牌时，模型也容易只删除目标卡牌，而没有充分表现玻璃桌面和其他卡牌在重力下重新稳定的结果。

这类失败的本质不是“没听懂要删什么”，而是没有把编辑动作解释成物理干预，也没有推理干预后的稳定状态。因此，causal_settle_route 应引入 CLEVRER-style causal decomposition，用于 Mechanics/Causality-heavy tasks。

适用范围：

适用	不优先适用
删除支撑物、移除接触点、改变承重关系、让物体倒下/滑落/倾斜、因刺激改变主体行为	纯反射、纯折射、普通阴影重投影、全局天气/季节变化

分解流程：

descriptive

-> 当前有哪些物体、支撑、接触、遮挡和稳定关系？

explanatory

-> 当前状态为什么稳定？哪个物体提供支撑或约束？

counterfactual / intervention

-> 用户的 edit_operation 删除/移动/添加了哪个因果因素？

predictive

-> 干预后系统应达到什么最终稳定状态？

visual realization

-> 最终状态在图像中应表现为哪些姿态、接触、阴影、遮挡和背景修复？

verification

-> 检查干预对象是否消失/改变，且反事实后果是否出现。

对 Planner 的要求：

```
{
  "route": "causal_settle_route",
  "reasoning_type": ["descriptive", "explanatory", "counterfactual", "predictive"],
  "physical_operation": "remove_support_and_settle",
  "current_state": {
    "support_relations": [],
    "contact_points": [],
    "stable_reason": ""
  },
  "intervention": {
    "operation": "remove",
    "removed_causal_factor": ""
  },
  "predicted_outcome": {
    "final_pose": "",
    "new_contact_points": [],
    "secondary_object_changes": [],
    "visual_effects": []
  },
  "must_pass_checks": []
}
```

摩托车 kickstand 例子：

descriptive: 白色摩托车当前直立，侧脚撑脚垫接触地面。

explanatory: 侧脚撑提供侧向支撑，使摩托车不会向左倒下。

counterfactual: 如果移除侧脚撑，原来的支撑关系消失。

predictive: 摩托车应在重力下倒向左侧，最终左侧车身/发动机区域接触路面。

visual realization: 删除脚撑和旧接触痕迹，修复路面；生成倒伏姿态、轮胎角度变化、左侧连续接触阴影。

verification: kickstand absent; motorcycle not upright; left-side ground contact visible; continuous contact shadow present。

卡牌支撑玻璃桌例子：

descriptive: 玻璃桌角由竖直卡牌支撑，卡牌与桌面/地面形成接触。

explanatory: 竖直卡牌承担局部载荷，维持玻璃桌面近似水平。

counterfactual: 如果移除前左侧支撑卡牌，该角点失去支撑。

predictive: 玻璃桌应向失去支撑的一侧下沉或倾斜，附近卡牌可能倒下并在地面形成新接触。

visual realization: 被删卡牌消失，玻璃角下沉，至少一个受影响卡牌倒伏，阴影/反射/遮挡随新姿态更新。

verification: removed card absent; visible support gap; glass tilted toward removed support; fallen card grounded; new shadows/reflections consistent。

对 Router 的要求：

```
if physics_law == "Causality"
  and edit_operation in {"remove", "move"}
  and task profile mentions support/contact/load/balance:
    route = "causal_settle_route"
    retry_budget = high
    verifier_focus = intervention + predicted_outcome
```

对 Verifier 的要求：

causal_settle_route 的 pass 条件必须同时包含两部分：

- 1. 干预对象是否完成：例如 kickstand/card 是否真的被删除。
- 2. 反事实后果是否完成：例如摩托车是否倒伏、玻璃是否倾斜、是否有新的接触阴影。

只完成第一部分不能 pass。这一点用于修正当前 MVP 中“支撑物被删了，但受影响主体仍保持原状态也被接受”的问题。

实现优先级：

- 1. 先在 Planner prompt 中加入 causal decomposition scaffold。
- 2. 再让 Router 对 Causality + remove/move + support/contact 强制选择 causal_settle_route。
- 3. 然后为该 route 增加专项 verifier checklist。
- 4. 最后才考虑接入检测/分割工具，用于定位支撑点、接触区域和受影响物体。

Executor

Executor 的职责是执行 Router 给出的 RouteDecision，把 TaskProfile 中的目标、依赖区域和 must-pass checks 转成一次或多次图像编辑调用。它不应自行重新判断物理类型；物理策略来自 Planner/Router，Executor 负责可靠执行、保存中间产物和把失败信息交回 Retry。

输入与输出：

```
{
  "inputs": {
    "canonical_image": "canonical_input.png",
    "task_profile": {},
    "route_decision": {},
    "coordinate_transform": {},
    "available_tools": {}
  },
  "outputs": {
    "candidate_images": [],
    "execution_trace": [],
    "intermediate_artifacts": [],
    "tool_errors": [],
    "selected_candidate": ""
  }
}
```

Executor 的核心优化方向：

优化点	当前 MVP	下一步方案
执行粒度	单次 whole-image edit	支持 route-specific one-pass / two-pass / local-edit / best-of-N
工具输入	只使用 image + prompt	使用 canonical image、target/dependency/preserve masks、reference cues
中间状态	只保存 candidate.png	保存每步输入图、mask、prompt、tool response、候选图和失败原因
失败处理	Verifier 失败后重跑同一路线	根据失败类型切换 route hint、扩大 mask、提高候选数或回退
非编辑区保护	依赖模型自觉保持	使用 preserve mask、局部编辑、PSNR/SSIM guardrail

按 route 的执行策略：

Route	Executor 执行方式	为什么这样执行
direct_global_edit	单次整图编辑；允许大范围风格/状态变化；重点保存结构身份	天气、季节、昼夜变化必须全图一致，局部编辑会破坏整体状态
localized_inpaint	用 target_mask 做局部删除/替换，再检查背景连续性	小目标任务需要减少非编辑区漂移
reflection_add_or_move_route	两阶段优先：先保证主体在目标区域可见，再补反射、水线、微波纹；高风险时 best-of-N	反射任务失败常来自主体位置不准或倒影缺失，分阶段更容易定位失败
reflection_remove_route	同时编辑 target_mask 和 dependency_mask；删除主体、倒影、高光和水面/镜面扰动	只删主体会留下镜像或高光残影

Route	Executor 执行方式	为什么这样执行
shadow_projection_route	编辑主体后重建 cast shadow/contact shadow；必要时第二步专门修阴影	阴影常位于主体外部，单次 prompt 容易漏掉或方向错误
light_source_route	添加/替换光源后，第二步调整周围受光面、色温、阴影和反射	光源影响范围超过灯本体，通常是半全局效应
refraction_reconstruction_route	用 dependency region 重建透明介质后的背景，同时清理 caustic、边缘高光、扭曲纹理	折射问题的关键是被介质扭曲的背景，不是透明物本体
causal_settle_route	先执行干预对象变化，再执行 predicted stable outcome；若无法局部控制则一次 prompt 强制描述最终稳定态	删除支撑物后必须改变受影响主体的姿态和接触关系
deformation_route	对形变区域做局部编辑，保护刚性连接和纹理连续区域	形变需要边界受控，避免整图结构漂移
localized_state_route	用局部 mask 改变材料状态，并保留周围上下文；边界不清时扩大一点 dependency mask	湿、冻、烧、融化等局部状态变化需要范围可控

Executor 需要支持三种执行模式：

```
one_pass:
  route 低风险或全局任务，直接调用 image edit API。

two_pass:
  第一步完成主体编辑，第二步补物理依赖，如 reflection/shadow/light spill。

best_of_n:
  对高风险 route 生成多个候选，用 verifier/candidate_ranker 选分数最高者。
```

route 到执行模式的默认映射：

Route	默认模式	高风险时
direct_global_edit	one_pass	best_of_n=2
localized_inpaint	one_pass_local	扩大 mask 后重试
reflection_add_or_move_route	two_pass	best_of_n=3
shadow_projection_route	two_pass	第二步专门修阴影
causal_settle_route	one_pass_with_strong_final_state	best_of_n=3 或 two-pass
localized_state_route	one_pass_local	mask 边界调整后重试

当前闭源 API 仍以 whole-image edit 为主，因此短期实现时可以先模拟 route-specific execution：

```
route-specific prompt template
+ canonical input
+ route-specific verifier focus
+ saved execution trace
```

等 mask/local edit 工具接入后，再把相同 route 切换到真正的局部执行。这样可以先验证 route 策略是否提升低分类型，再增加工具复杂度。

Executor 必须保存执行 trace，方便科研汇报和失败复盘：

```
{
  "step": 0,
  "route": "causal_settle_route",
  "mode": "one_pass_with_strong_final_state",
  "input_image": "canonical_input.png",
  "prompt": "...",
  "masks": {
    "target_mask": null,
    "dependency_mask": null,
    "preserve_mask": null
  },
  "candidate_image": "candidate.png",
  "known_limitations": ["no local mask tool available"]
}
```

Executor 的短期开发顺序：

- 1. 先把 execute_edit 包装为 route-aware executor，保存 execution_trace.json。
- 2. 为 Reflection、Light Propagation、Causality 三类低分 route 写 route-specific prompt template。
- 3. 为高风险 route 加 best_of_n 接口，但默认 n=1，避免成本失控。

- 4. 接入 mask 协议后，先支持 `target_mask` 和 `dependency_mask` 的本地文件输入。
- 5. 最后再接真实 `detector/segmenter/inpainter`。

Verifier

Verifier 的优化目标是从“通用 VLM 总分判断”升级为“route-aware gate”。它应同时使用完整图、必要的 crop/局部证据、TaskProfile 的 `must_pass_checks`、Router 的 `verification_focus` 和 PICABench 的 QA 视图分流结果。

输入：

```
{
  "original_image": "canonical_input.png",
  "candidate_image": "candidate.png",
  "task_profile": {},
  "route_decision": {},
  "mapped_qa_views": [],
  "coordinate_transform": {}
}
```

输出：

```
{
  "pass": false,
  "instruction_score": 0,
  "preservation_score": 0,
  "physics_score": 0,
  "check_results": [],
  "blocking_failures": [],
  "issues": [],
  "repair_instruction": "",
  "route_hints": []
}
```

三类标签在 Verifier 中的作用：

标签	Verifier 检查方式
physics_category	选择大类评分口径：Optics 检查光照依赖，Mechanics 检查接触/稳定，State 检查状态范围
physics_law	选择 law-specific checklist，例如 Reflection 检查主体和反射是否同时存在、尺度是否一致
edit_operation	检查动作是否真的发生，例如 add 后目标可见，remove 后目标消失，move 后旧位置清理且新位置正确

新 crop 协议在 Verifier 中的用法：

问题类型	评估视图	Verifier 用途
global_position	完整候选图	判断目标在画布中的真实方位、透视、前后景、左右上下关系
local_appearance	mapped crop	判断阴影、反射、接触、材质、纹理、折射等局部物理证据
mixed	完整图 + mapped crop	完整图判断位置，crop 判断局部物理细节
unknown	mapped crop + context_risk	结果单独标记风险，不作为强结论直接解释

示例 checklist：

Law	Operation	必要检查
Reflection	add	主体可见；反射可见；反射位于镜面/水面正确区域；尺度/方向合理；接触线或微波纹存在
Light_Propagation	move	目标新位置正确；旧阴影不存在；新阴影方向与参考阴影一致；阴影软硬与场景匹配
Causality	remove	被删支撑不存在；受影响物体不悬空；最终姿态稳定；新接触阴影存在
Local	wet	局部区域变湿；边界不越界；材质变暗/反光；周围未被全局改写

Verifier 的 pass 不应只依赖总分，而应要求：

```
pass = no blocking_failures
      and instruction_score >= threshold
      and preservation_score >= threshold
      and physics_score >= threshold
      and required must_pass_checks all pass
```

对低分类型的专项要求：

Route	Verifier 必须阻断的失败
reflection_add_or_move_route	主体不可见；反射不可见；反射没有与主体/水线对齐；只有主体没有倒影
shadow_projection_route	移动物体后没有新阴影；旧阴影残留；阴影方向与参考阴影明显冲突
causal_settle_route	只删除支撑物但受影响主体仍保持原稳定状态；没有新接触点或接触阴影
localized_state_route	局部状态扩散到无关区域；材料边界不合理

因此，新 Verifier 的核心不是“给更严格的自然语言评价”，而是把 `must_pass_checks` 变成可解释的 blocking gate。只要 blocking check 失败，即使 VLM 给出较高总分，也不能 pass。

PICAEval

PICAEval 使用三类标签做两件事：评估输入选择和结果归因。

标签	PICAEval 用途
physics_category	按 Optics/Mechanics/State 汇总，观察大类能力
physics_law	按具体 law 汇总，定位最低分机制，例如 Reflection、Light_Propagation
edit_operation	与 law 交叉统计，找出最难组合，例如 Reflection + add、Light_Propagation + move

当前已经实现的 QA 视图分流也应与标签结合：

```
位置/构图类 QA -> full image
局部物理属性 QA -> mapped crop
混合 QA -> full image + mapped crop
```

后续汇总指标建议增加：

```
accuracy_by_category
accuracy_by_law
accuracy_by_operation
accuracy_by_law_operation
context_risk_accuracy
```

Retry Loop

Retry 不应只把自然语言 `repair_instruction` 拼回 prompt，而应根据三类标签把失败转成下一轮 route hints。

失败类型	标签组合	下一轮策略
目标没出现	add	提高目标可见性约束，先执行主体插入，再补物理效应
反射缺失	Reflection + add/move	强制进入 reflection route，补 reflected region
阴影方向错误	Light_Propagation	选择参考阴影，重写 shadow projection prompt
支撑移除后物体没倒	Causality + remove	将操作改写为 <code>remove_support_and_settle</code> ，要求最终稳定态
局部状态扩散过大	Local	缩小 edit scope，优先 mask/local route

因此，三类标签在 retry 中的作用是把失败从“自然语言建议”升级为“下一轮路由和工具选择的结构化条件”。

标签驱动规划与路由的参考工作

以下论文/项目可作为本项目利用 `physics_category`、`physics_law`、`edit_operation` 进行 Planner、Router、Verifier 设计的参考。

参考工作	可借鉴点	对本项目的对应关系
PICABench / PICAEval	用物理 taxonomy 和 region-grounded QA 评估物理一致性	直接对应 <code>physics_category</code> 、 <code>physics_law</code> 和 law-specific verifier checklist
PHYRE	用目标状态、动作干预和物理模拟结果定义任务	对应 <code>edit_operation</code> 作为干预， <code>expected_stable_outcome</code> 作为编辑后目标状态
Forward Prediction for Physical Reasoning	将 forward prediction model 接入 physical-reasoning agent，说明预测模块可提升复杂物理任务表现	支持在 Causality/Deformation route 中加入“后果预测”或稳定态推理模块
Physion	强调 object-centric、物理有意义的场景表示，而不只是像素预测	支持 Planner 输出 <code>target_objects</code> 、 <code>affected_objects</code> 、 <code>dependent_regions</code>
CLEVRER	将动态场景问题拆成描述、解释、预测、反事实	支持把 <code>edit_operation</code> 看作干预，把 <code>physics_law</code> 看作因果机制，把 verifier 看作后果检查

参考工作	可借鉴点	对本项目的对应关系
IntPhys	用违反预期原则检查 object permanence、continuity、solidity 等物理一致性	支持把每个 law 转成 must-pass violation checks
Chameleon	LLM planner 根据任务约束组合工具模块	支持 Router 不由 LLM 自由决定，而由任务约束和工具 inventory 共同决定
HuggingGPT	将复杂任务拆成 task planning、model selection、execution、response generation	对应本项目 Planner -> Router -> Executor -> Verifier 的模块化结构
ViperGPT	用 LLM 生成程序组合视觉模块，提升可解释性和可执行性	支持将 route 表达成可执行工具链，而不是纯自然语言计划
Visual ChatGPT	用 ChatGPT 管理视觉基础模型和多步视觉编辑反馈	支持图像编辑 agent 中的工具描述、视觉反馈和多轮修复流程

需要注意的是，这些工作没有一个可以直接照搬到物理一致性图像编辑中。最接近任务本身的是 PICABench；PHYRE、Physion、CLEVRER、IntPhys 提供物理任务分解和评估思想；Chameleon、HuggingGPT、ViperGPT、Visual ChatGPT 提供 planner/router/tool-use 的工程形态。对本项目最合理的融合方式是：用 PICABench 的三类标签定义物理任务语义，用物理推理 benchmark 的机制/干预/后果分解指导 TaskProfile，用 tool-use agent 的模块化思想实现 route 和 verifier。